

```
In [ ]: !pip install einops
        !pip install torchinfo
        !pip install datasets
```

HW8: Diffusions & Low-Rank Adaptation (LoRA)

In this assignment, you will learn to implement diffusion model and low-rank adaptation (LoRA) from scratch using the Pytorch library.

This homework consists of two main sections:

In the first section, we introduce the diffusion model where you will be tasked to implement various components for training diffusion model, including a noise scheduler, sampling module, and model architectures. After building our these compoments, we will assemble them and train the diffusion model on the MNIST dataset.

The second part will introduce you to parameter-efficient transfer learning (PET) where LoRA will be used to transfer the MaskFormer (<https://arxiv.org/abs/2107.06278>), an instance segmentation model, to semantic segmentation task on a satellite-building segmentation dataset. This section will teach you how LoRA works and how to implement it from scratch using `forward_hook`.

In this section, you will be tasked to implement each component of the diffusion model (DDPM).

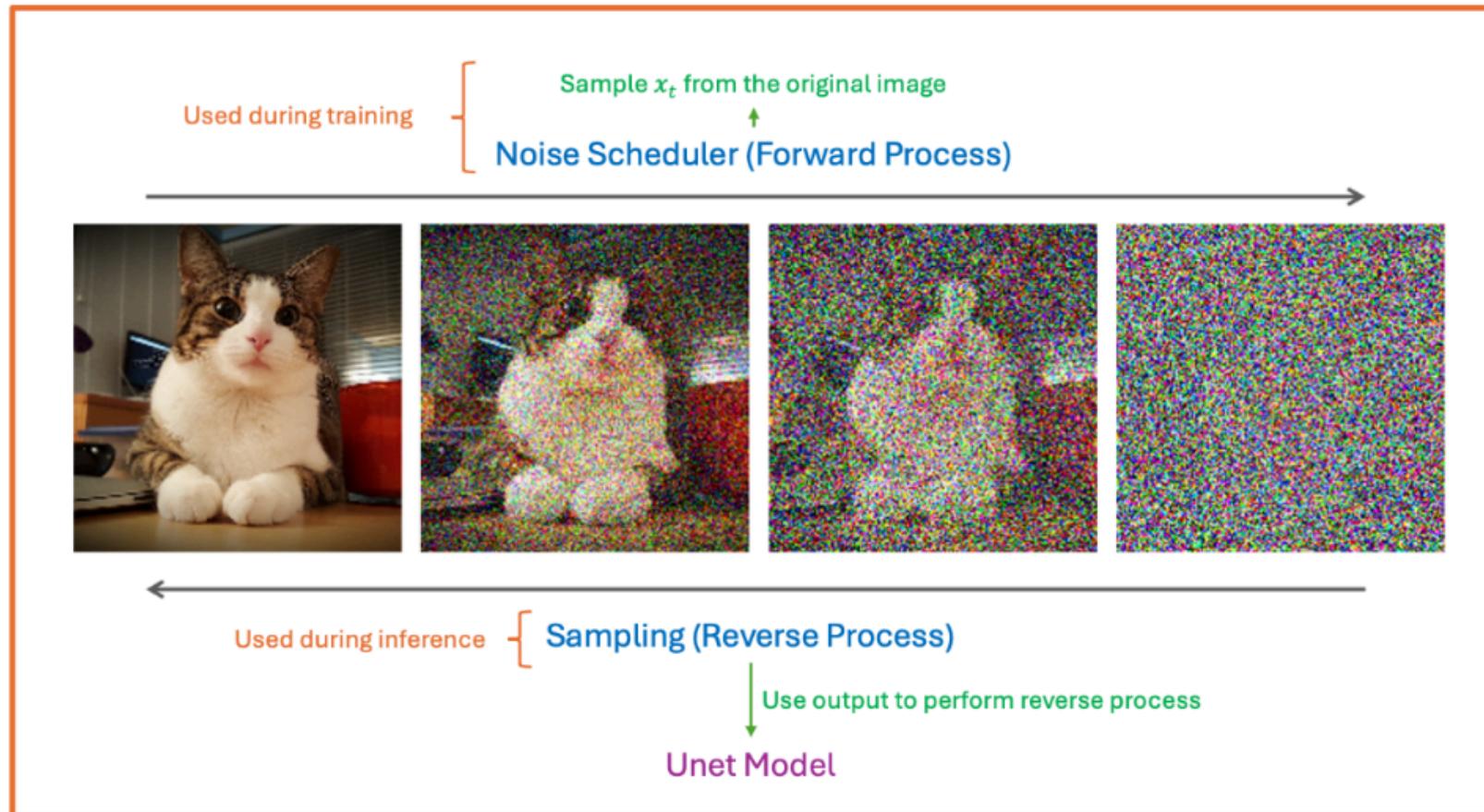
Based on the components in DDPM, this section is organized into four parts :

1. **Noise Scheduler (Foward Process):** During diffusion's forward process, the noise was gradually added, transforming the clean image x_0 into noisy image x_t w.r.t to the timestep t . To do so, the noise scheduler is used for controlling the amount of noise to be added in each timestep.
2. **Sampling (Reverse Process):** During the revesere process, the sampling algorithm is employed to update x_t to remove the noise inside it using the noise estimation model θ .
3. **Model Architecture:** The deep learning model θ is for predicting the noise in x_t to be used in reverse process. In this homework, we will use UNet as a base architecture.

4. **Training Process:** The training process will combine these three modules to perform a learning process.

The overview of the system is shown below.

Diffusion Model



Downloading CIFAR10 Dataset

The MNIST dataset comprises of 60,000 training images at 32x32 resolution; the images are normalized to $[-1, 1]$. In the following, you will learn how to train diffusion on this dataset.

```
In [2]: import os
import torch
import torchvision.datasets as datasets
from torch.utils.data import DataLoader
from torchvision.transforms import Compose, ToTensor, Resize
from einops import rearrange
from tqdm import tqdm
# from torch_ema import ExponentialMovingAverage

class Rescale(object):
    def __init__(self, old_range, new_range):
        self.old_range = old_range
        self.new_range = new_range

    def __call__(self, image):
        old_min, old_max = self.old_range
        new_min, new_max = self.new_range
        image -= old_min
        image *= (new_max - new_min) / (old_max - old_min)
        image += new_min
        return image

normalize_to_neg_one_to_one = Rescale((0, 1), (-1, 1))
unnormalize_to_zero_to_one = Rescale((-1, 1), (0, 1))

RESOLUTION = (32, 32)
BATCH_SIZE = 64

transform = Compose([
    Resize(RESOLUTION), lambda x: x.convert('RGB'), ToTensor(), normalize_to_neg_one_to_one
])

training_data = datasets.MNIST(
    root="./data",
    train=True,
    download=True,
    transform=transform
)

test_data = datasets.MNIST(
    root="./data",
```

```

    train=False,
    download=True,
    transform=transform
)

train_dataloader = DataLoader(training_data, batch_size=BATCH_SIZE, shuffle=True)
test_dataloader = DataLoader(test_data, batch_size=BATCH_SIZE, shuffle=False)

```

These are example images from the CIFAR dataset.

```

In [3]: import matplotlib.pyplot as plt
import numpy as np

fig, axs = plt.subplots(ncols=9, figsize=(15, 75))
for i in range(9):
    image, label = training_data[i]
    axs[i].imshow(rearrange(unnormalize_to_zero_to_one(image), "c h w -> h w c"))
    axs[i].set_title(training_data.classes[label])
    axs[i].set_axis_off()
plt.show()

```



Plot training and validation loss

Loss of the last epoch should be around 0.0085 - 0.009.

```

In [ ]: plt.plot(range(epoch), train_losses, label="training loss")
plt.plot(range(epoch), val_losses, label="validation loss")
plt.legend()
plt.show()

```

Inference

```
In [ ]: trainer.eval()
fig, axs = plt.subplots(nrows=10, ncols=10)
for digit in range(10):
    label = torch.ones(10, device=trainer.device, dtype=torch.int) * digit
    context = trainer.embedding(label).unsqueeze(1)
    sampled_images = trainer.model.sample(batch_size = 10, context=context)
    for i in range(10):
        axs[digit][i].imshow(rearrange(unnormalize_to_zero_to_one(sampled_images[i]), "c h w -> h w c"), cr
        # axs[digit].set_title(label[0].item())
        axs[digit][i].set_axis_off()
plt.show()
```

Part 2: Low Rank Adapter (LORA)

With the discovery of the scaling property in the deep learning model, several researchers tend to increase the size of the deep learning model to obtain the emergent property, especially in the natural language processing (NLP) field. For example, the GPT3 contains 175 billion parameters, making it impossible to fine-tune. This trend obstructs students like us from transferring these enormous foundation models on a single GPU (or small resources). Therefore, to alleviate this problem, some researchers invent new methods to fine-tune the model, called parameter-efficient transfer learning, which allows us to train large models on limited resources. Its benefits exist not only in the training process but also during deployment. After fine-tuning the model, we need to save only a small amount of parameters (LoRA weights), allowing us to deploy the foundation model to various downstream tasks using a small amount of storage. One of the prevailing methods is Low Rank Adaptation (LORA).

In this section, we are going to introduce the Low-Rank Adaptation. You are assigned to implement the LORA on the MaskFormer model trained by HuggingFace. The model is trained on ADE20k semantic segmentation, and we will transfer this foundation model to the satellite building segmentation dataset using the LoRA method. For the details of MaskFormer, please visit this site: <https://huggingface.co/facebook/maskformer-swin-tiny-ade>.

Downloading the pre-trained MaskFormer model and Satellite-Building-Segmentation Dataset

```
In [1]: from PIL import Image, ImageDraw
import torch.nn as nn
```

```

import torch
import torch.nn.functional as F
from transformers import MaskFormerConfig, MaskFormerModel, MaskFormerImageProcessor, MaskFormerForInstanceSegmentation

from datasets import load_dataset

model_name = "facebook/maskformer-swin-tiny-ade"
ds = load_dataset("kerememberke/satellite-building-segmentation", revision="refs/convert/parquet")

```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Preprocessing Data

```

In [2]: def create_mask(data):
        image = data["image"]
        mask = Image.new(mode="RGB", size=image.size, color=(0, 0, 0))
        draw = ImageDraw.Draw(mask)

        instance_id = 1
        for category_id, polygon_point in zip(data["objects"]["category"], data["objects"]["segmentation"]):
            if len(polygon_point) > 1:
                print(polygon_point)
                raise IndexError
            draw.polygon(polygon_point[0], fill=(category_id+1, instance_id, 0))
            instance_id += 1
        return mask

def unnormalize_image(image, mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]):
    image = image * np.array([0.229, 0.224, 0.225][:, None, None])
    image = image + np.array([0.485, 0.456, 0.406][:, None, None])
    return image

```

```

In [3]: from torch.utils.data import DataLoader, Dataset
import albumentations as A
import numpy as np
from PIL import Image, ImageDraw
import matplotlib.pyplot as plt
from einops import rearrange

```

```

class ImageSegmentationDataset(Dataset):
    def __init__(self, dataset, processor, transform=None):
        self.dataset = dataset
        self.processor = processor
        self.transform = transform

    def __len__(self):
        return len(self.dataset)

    def __getitem__(self, idx):
        # Convert the PIL Image to a NumPy array
        image = np.array(self.dataset[idx]["image"].convert("RGB"))

        # Get the pixel wise instance id and category id maps
        # of shape (height, width)
        instance_seg = np.array(create_mask(self.dataset[idx]))[..., 1]
        class_id_map = np.array(create_mask(self.dataset[idx]))[..., 0]
        class_labels = np.unique(class_id_map)
        # Build the instance to class dictionary
        inst2class = {}
        for label in class_labels:
            instance_ids = np.unique(instance_seg[class_id_map == label])
            inst2class.update({i: label for i in instance_ids})
        # Apply transforms
        if self.transform is not None:
            transformed = self.transform(image=image, mask=instance_seg)
            (image, instance_seg) = (transformed["image"], transformed["mask"])

            # Convert from channels last to channels first
            image = image.transpose(2,0,1)
        if class_labels.shape[0] == 1 and class_labels[0] == 0:
            # If the image has no objects then it is skipped
            inputs = self.processor([image], return_tensors="pt")
            inputs = {k:v.squeeze() for k,v in inputs.items()}
            inputs["class_labels"] = torch.tensor([0])
            inputs["mask_labels"] = torch.zeros(
                (0, inputs["pixel_values"].shape[-2], inputs["pixel_values"].shape[-1])
            )
        else:
            # Else use process the image with the segmentation maps
            inputs = self.processor(
                [image],

```

```

        [instance_seg],
        instance_id_to_semantic_id=inst2class,
        return_tensors="pt"
    )
    inputs = {
        k:v.squeeze() if isinstance(v, torch.Tensor) else v[0] for k,v in inputs.items()
    }
    # Return the inputs
    segmentation_mask = (inputs["mask_labels"] * inputs["class_labels"][:, None, None]).sum(dim=0).clamp_
    inputs["segmentation"] = segmentation_mask
    return inputs

train_val_transform = A.Compose([
    A.Resize(width=128, height=128),
])

processor = MaskFormerImageProcessor.from_pretrained(model_name)
processor.size = {'shortest_edge': 128, 'longest_edge': 2048}

train_dataset = ImageSegmentationDataset(
    ds["train"],
    processor=processor,
    transform=train_val_transform
)
val_dataset = ImageSegmentationDataset(
    ds["validation"],
    processor=processor,
    transform=train_val_transform
)
test_dataset = ImageSegmentationDataset(
    ds["test"],
    processor=processor,
    transform=train_val_transform
)

```

```

In [4]: for idx, data in enumerate(train_dataset):
        image = unnormalize_image(data["pixel_values"])
        fig, axs = plt.subplots(ncols=2, figsize=(8, 5))
        axs[0].imshow(np.array(rearrange(image, "c h w -> h w c")))
        axs[1].imshow(np.array(data["segmentation"]))
        axs[0].set_axis_off()
        axs[1].set_axis_off()

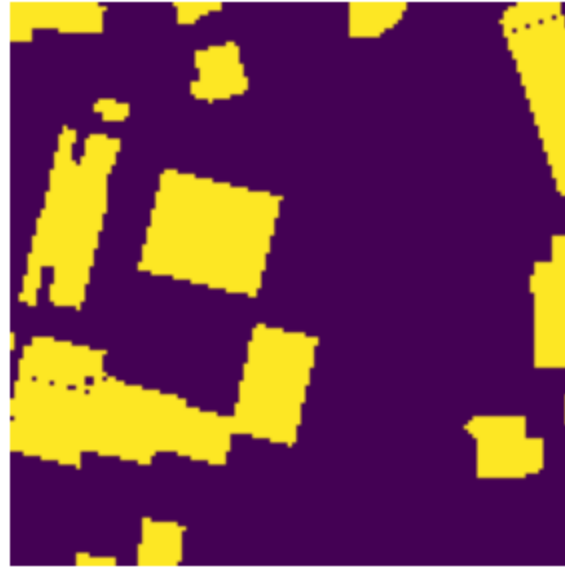
```

```
plt.show()
```

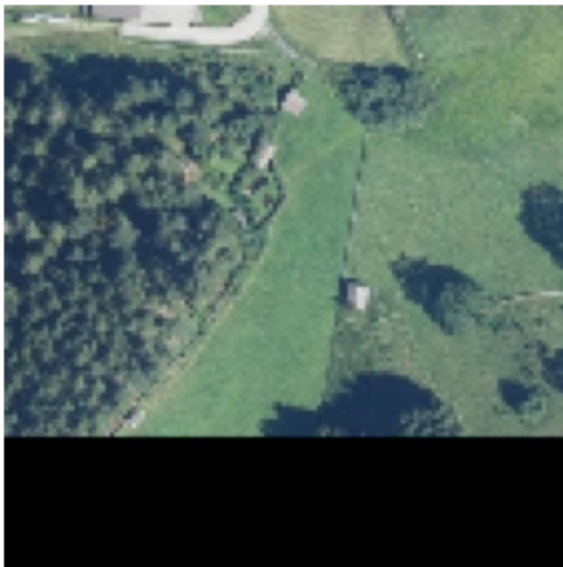
```
if idx == 10:  
    break
```

```
/tmp/ipykernel_180391/1125473636.py:16: DeprecationWarning: __array_wrap__ must accept context and return_s  
calar arguments (positionally) in the future. (Deprecated NumPy 2.0)  
    image = image * np.array([0.229, 0.224, 0.225])[:, None, None]  
/tmp/ipykernel_180391/1125473636.py:17: DeprecationWarning: __array_wrap__ must accept context and return_s  
calar arguments (positionally) in the future. (Deprecated NumPy 2.0)  
    image = image + np.array([0.485, 0.456, 0.406])[:, None, None]  
/tmp/ipykernel_180391/1490522105.py:4: DeprecationWarning: __array__ implementation doesn't accept a copy k  
eyword, so passing copy=False failed. __array__ must implement 'dtype' and 'copy' keyword arguments. To lea  
rn more, see the migration guide https://numpy.org/devdocs/numpy\_2\_0\_migration\_guide.html#adapting-to-changes-in-the-copy-keyword  
    axs[0].imshow(np.array(rearrange(image, "c h w -> h w c")))  
/tmp/ipykernel_180391/1490522105.py:5: DeprecationWarning: __array__ implementation doesn't accept a copy k  
eyword, so passing copy=False failed. __array__ must implement 'dtype' and 'copy' keyword arguments. To lea  
rn more, see the migration guide https://numpy.org/devdocs/numpy\_2\_0\_migration\_guide.html#adapting-to-changes-in-the-copy-keyword  
    axs[1].imshow(np.array(data["segmentation"]))
```



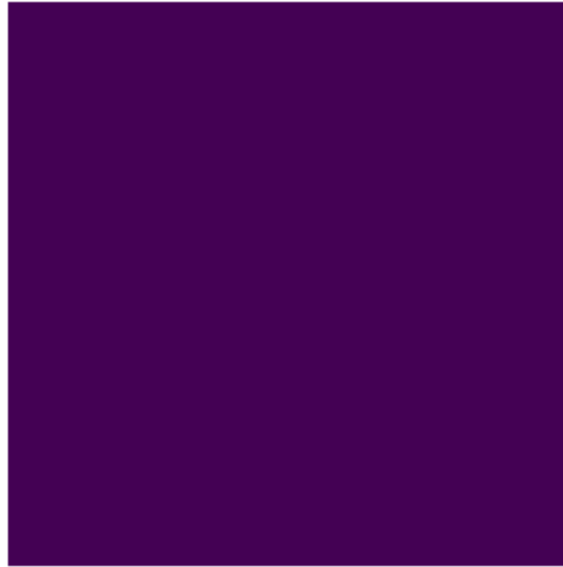


Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-2.288818357065736e-08..0.9411764984130859].

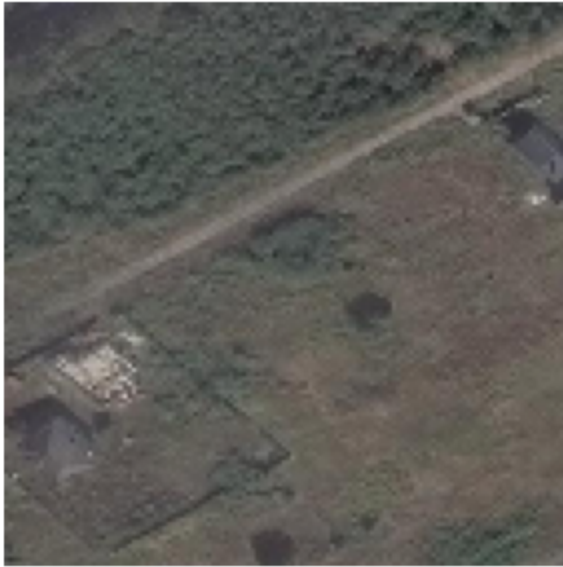


Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-2.288818357065736e-08..0.8078431396484376].







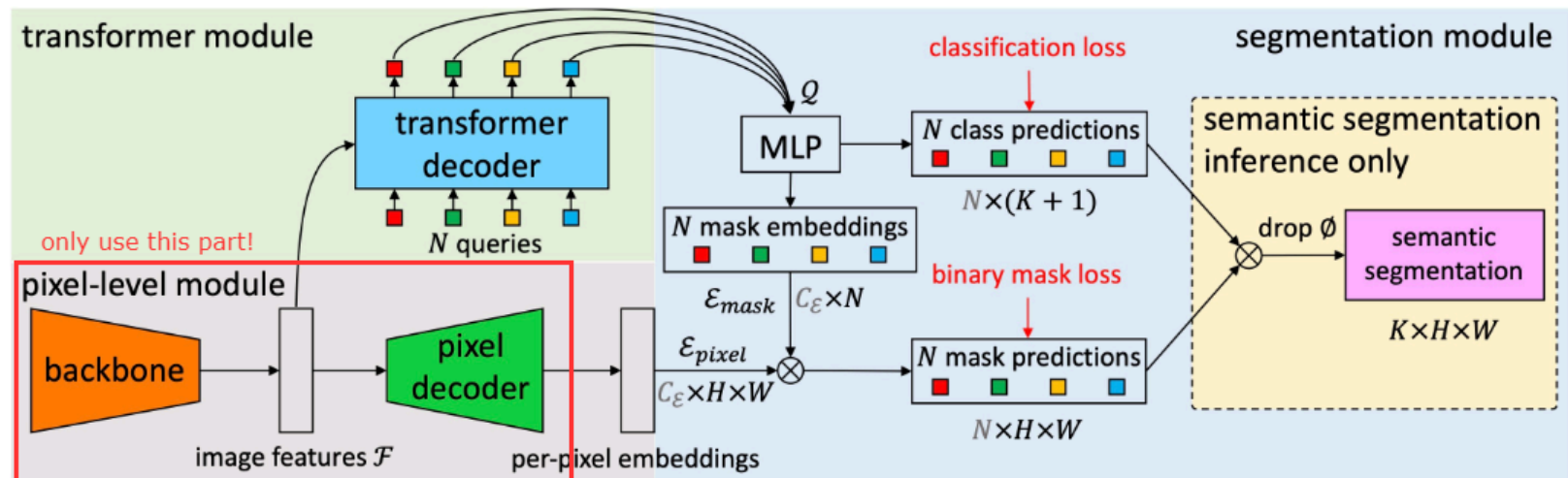


```
In [5]: def collate_fn(examples):  
        # Get the pixel values, pixel mask, mask labels, and class labels  
        pixel_values = torch.stack([example["pixel_values"] for example in examples])  
        pixel_mask = torch.stack([example["pixel_mask"] for example in examples])  
        segmentation = torch.stack([example["segmentation"] for example in examples])
```

```
mask_labels = [example["mask_labels"] for example in examples]
class_labels = [example["class_labels"] for example in examples]
# Return a dictionary of all the collated features
return {
    "pixel_values": pixel_values,
    "pixel_mask": pixel_mask,
    "segmentation": segmentation,
    "mask_labels": mask_labels,
    "class_labels": class_labels
}
# Building the training and validation dataloader
train_dataloader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True,
    collate_fn=collate_fn
)
val_dataloader = DataLoader(
    val_dataset,
    batch_size=32,
    shuffle=False,
    collate_fn=collate_fn
)
test_dataloader = DataLoader(
    test_dataset,
    batch_size=32,
    shuffle=False,
    collate_fn=collate_fn
)
```

Model extraction

After preprocessing the dataset, we are going to extract pixel-level module from the MaskFormer model to perform semantic segmentation. Specifically, we will use only module framed inside the red rectangle.



Hint: You can use `model.named_modules()` to observe modules inside the model.

```
In [6]: maskformer = MaskFormerForInstanceSegmentation.from_pretrained(model_name)
for name, module in maskformer.named_modules():
    print(name)
```

Loading weights: 0% | 0/432 [00:00<?, ?it/s]

```
model
model.pixel_level_module
model.pixel_level_module.encoder
model.pixel_level_module.encoder.model
model.pixel_level_module.encoder.model.embeddings
model.pixel_level_module.encoder.model.embeddings.patch_embeddings
model.pixel_level_module.encoder.model.embeddings.patch_embeddings.projection
model.pixel_level_module.encoder.model.embeddings.norm
model.pixel_level_module.encoder.model.embeddings.dropout
model.pixel_level_module.encoder.model.encoder
model.pixel_level_module.encoder.model.encoder.layers
model.pixel_level_module.encoder.model.encoder.layers.0
model.pixel_level_module.encoder.model.encoder.layers.0.blocks
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.layer_norm_before
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.attention
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.attention.self
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.attention.self.query
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.attention.self.key
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.attention.self.value
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.attention.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.attention.output
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.attention.output.dense
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.attention.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.drop_path
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.layer_norm_after
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.intermediate
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.intermediate.dense
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.intermediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.output
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.output.dense
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.0.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.layer_norm_before
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.attention
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.attention.self
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.attention.self.query
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.attention.self.key
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.attention.self.value
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.attention.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.attention.output
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.attention.output.dense
```

```
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.attention.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.drop_path
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.layer_norm_after
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.intermediate
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.intermediate.dense
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.intermediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.output
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.output.dense
model.pixel_level_module.encoder.model.encoder.layers.0.blocks.1.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.0.downsample
model.pixel_level_module.encoder.model.encoder.layers.0.downsample.reduction
model.pixel_level_module.encoder.model.encoder.layers.0.downsample.norm
model.pixel_level_module.encoder.model.encoder.layers.1
model.pixel_level_module.encoder.model.encoder.layers.1.blocks
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.layer_norm_before
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.attention
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.attention.self
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.attention.self.query
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.attention.self.key
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.attention.self.value
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.attention.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.attention.output
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.attention.output.dense
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.attention.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.drop_path
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.layer_norm_after
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.intermediate
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.intermediate.dense
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.intermediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.output
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.output.dense
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.0.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.layer_norm_before
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.attention
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.attention.self
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.attention.self.query
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.attention.self.key
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.attention.self.value
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.attention.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.attention.output
```

```
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.attention.output.dense
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.attention.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.drop_path
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.layer_norm_after
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.intermediate
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.intermediate.dense
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.intermediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.output
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.output.dense
model.pixel_level_module.encoder.model.encoder.layers.1.blocks.1.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.1.downsample
model.pixel_level_module.encoder.model.encoder.layers.1.downsample.reduction
model.pixel_level_module.encoder.model.encoder.layers.1.downsample.norm
model.pixel_level_module.encoder.model.encoder.layers.2
model.pixel_level_module.encoder.model.encoder.layers.2.blocks
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.layer_norm_before
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.attention
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.attention.self
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.attention.self.query
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.attention.self.key
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.attention.self.value
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.attention.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.attention.output
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.attention.output.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.attention.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.drop_path
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.layer_norm_after
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.intermediate
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.intermediate.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.intermediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.output
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.output.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.0.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.layer_norm_before
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.attention
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.attention.self
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.attention.self.query
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.attention.self.key
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.attention.self.value
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.attention.self.dropout
```

```
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.attention.output
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.attention.output.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.attention.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.drop_path
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.layer_norm_after
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.intermediate
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.intermediate.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.intermediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.output
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.output.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.1.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.layer_norm_before
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.attention
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.attention.self
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.attention.self.query
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.attention.self.key
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.attention.self.value
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.attention.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.attention.output
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.attention.output.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.attention.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.drop_path
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.layer_norm_after
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.intermediate
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.intermediate.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.intermediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.output
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.output.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.2.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.layer_norm_before
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.attention
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.attention.self
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.attention.self.query
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.attention.self.key
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.attention.self.value
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.attention.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.attention.output
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.attention.output.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.attention.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.drop_path
```

```
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.layer_norm_after
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.intermediate
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.intermediate.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.intermediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.output
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.output.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.3.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.layer_norm_before
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.attention
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.attention.self
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.attention.self.query
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.attention.self.key
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.attention.self.value
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.attention.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.attention.output
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.attention.output.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.attention.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.drop_path
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.layer_norm_after
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.intermediate
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.intermediate.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.intermediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.output
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.output.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.4.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.layer_norm_before
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.attention
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.attention.self
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.attention.self.query
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.attention.self.key
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.attention.self.value
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.attention.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.attention.output
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.attention.output.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.attention.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.drop_path
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.layer_norm_after
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.intermediate
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.intermediate.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.intermediate.intermediate_act_fn
```

```
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.output
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.output.dense
model.pixel_level_module.encoder.model.encoder.layers.2.blocks.5.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.2.downsample
model.pixel_level_module.encoder.model.encoder.layers.2.downsample.reduction
model.pixel_level_module.encoder.model.encoder.layers.2.downsample.norm
model.pixel_level_module.encoder.model.encoder.layers.3
model.pixel_level_module.encoder.model.encoder.layers.3.blocks
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.layer_norm_before
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.attention
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.attention.self
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.attention.self.query
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.attention.self.key
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.attention.self.value
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.attention.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.attention.output
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.attention.output.dense
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.attention.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.drop_path
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.layer_norm_after
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.intermediate
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.intermediate.dense
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.intermediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.output
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.output.dense
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.0.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.layer_norm_before
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.attention
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.attention.self
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.attention.self.query
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.attention.self.key
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.attention.self.value
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.attention.self.dropout
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.attention.output
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.attention.output.dense
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.attention.output.dropout
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.drop_path
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.layer_norm_after
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.intermediate
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.intermediate.dense
```

```
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.intermediate.intermediate_act_fn
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.output
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.output.dense
model.pixel_level_module.encoder.model.encoder.layers.3.blocks.1.output.dropout
model.pixel_level_module.encoder.model.layernorm
model.pixel_level_module.encoder.model.pooler
model.pixel_level_module.encoder.hidden_states_norms
model.pixel_level_module.encoder.hidden_states_norms.0
model.pixel_level_module.encoder.hidden_states_norms.1
model.pixel_level_module.encoder.hidden_states_norms.2
model.pixel_level_module.encoder.hidden_states_norms.3
model.pixel_level_module.decoder
model.pixel_level_module.decoder.fpn
model.pixel_level_module.decoder.fpn.stem
model.pixel_level_module.decoder.fpn.stem.0
model.pixel_level_module.decoder.fpn.stem.1
model.pixel_level_module.decoder.fpn.stem.2
model.pixel_level_module.decoder.fpn.layers
model.pixel_level_module.decoder.fpn.layers.0
model.pixel_level_module.decoder.fpn.layers.0.proj
model.pixel_level_module.decoder.fpn.layers.0.proj.0
model.pixel_level_module.decoder.fpn.layers.0.proj.1
model.pixel_level_module.decoder.fpn.layers.0.block
model.pixel_level_module.decoder.fpn.layers.0.block.0
model.pixel_level_module.decoder.fpn.layers.0.block.1
model.pixel_level_module.decoder.fpn.layers.0.block.2
model.pixel_level_module.decoder.fpn.layers.1
model.pixel_level_module.decoder.fpn.layers.1.proj
model.pixel_level_module.decoder.fpn.layers.1.proj.0
model.pixel_level_module.decoder.fpn.layers.1.proj.1
model.pixel_level_module.decoder.fpn.layers.1.block
model.pixel_level_module.decoder.fpn.layers.1.block.0
model.pixel_level_module.decoder.fpn.layers.1.block.1
model.pixel_level_module.decoder.fpn.layers.1.block.2
model.pixel_level_module.decoder.fpn.layers.2
model.pixel_level_module.decoder.fpn.layers.2.proj
model.pixel_level_module.decoder.fpn.layers.2.proj.0
model.pixel_level_module.decoder.fpn.layers.2.proj.1
model.pixel_level_module.decoder.fpn.layers.2.block
model.pixel_level_module.decoder.fpn.layers.2.block.0
model.pixel_level_module.decoder.fpn.layers.2.block.1
model.pixel_level_module.decoder.fpn.layers.2.block.2
```

```
model.pixel_level_module.decoder.mask_projection
model.transformer_module
model.transformer_module.position_embedder
model.transformer_module.queries_embedder
model.transformer_module.input_projection
model.transformer_module.decoder
model.transformer_module.decoder.layers
model.transformer_module.decoder.layers.0
model.transformer_module.decoder.layers.0.self_attn
model.transformer_module.decoder.layers.0.self_attn.k_proj
model.transformer_module.decoder.layers.0.self_attn.v_proj
model.transformer_module.decoder.layers.0.self_attn.q_proj
model.transformer_module.decoder.layers.0.self_attn.o_proj
model.transformer_module.decoder.layers.0.self_attn_layer_norm
model.transformer_module.decoder.layers.0.encoder_attn
model.transformer_module.decoder.layers.0.encoder_attn.k_proj
model.transformer_module.decoder.layers.0.encoder_attn.v_proj
model.transformer_module.decoder.layers.0.encoder_attn.q_proj
model.transformer_module.decoder.layers.0.encoder_attn.o_proj
model.transformer_module.decoder.layers.0.encoder_attn_layer_norm
model.transformer_module.decoder.layers.0.mlp
model.transformer_module.decoder.layers.0.mlp.fc1
model.transformer_module.decoder.layers.0.mlp.fc2
model.transformer_module.decoder.layers.0.mlp.activation_fn
model.transformer_module.decoder.layers.0.final_layer_norm
model.transformer_module.decoder.layers.1
model.transformer_module.decoder.layers.1.self_attn
model.transformer_module.decoder.layers.1.self_attn.k_proj
model.transformer_module.decoder.layers.1.self_attn.v_proj
model.transformer_module.decoder.layers.1.self_attn.q_proj
model.transformer_module.decoder.layers.1.self_attn.o_proj
model.transformer_module.decoder.layers.1.self_attn_layer_norm
model.transformer_module.decoder.layers.1.encoder_attn
model.transformer_module.decoder.layers.1.encoder_attn.k_proj
model.transformer_module.decoder.layers.1.encoder_attn.v_proj
model.transformer_module.decoder.layers.1.encoder_attn.q_proj
model.transformer_module.decoder.layers.1.encoder_attn.o_proj
model.transformer_module.decoder.layers.1.encoder_attn_layer_norm
model.transformer_module.decoder.layers.1.mlp
model.transformer_module.decoder.layers.1.mlp.fc1
model.transformer_module.decoder.layers.1.mlp.fc2
model.transformer_module.decoder.layers.1.mlp.activation_fn
```

```
model.transformer_module.decoder.layers.1.final_layer_norm
model.transformer_module.decoder.layers.2
model.transformer_module.decoder.layers.2.self_attn
model.transformer_module.decoder.layers.2.self_attn.k_proj
model.transformer_module.decoder.layers.2.self_attn.v_proj
model.transformer_module.decoder.layers.2.self_attn.q_proj
model.transformer_module.decoder.layers.2.self_attn.o_proj
model.transformer_module.decoder.layers.2.self_attn_layer_norm
model.transformer_module.decoder.layers.2.encoder_attn
model.transformer_module.decoder.layers.2.encoder_attn.k_proj
model.transformer_module.decoder.layers.2.encoder_attn.v_proj
model.transformer_module.decoder.layers.2.encoder_attn.q_proj
model.transformer_module.decoder.layers.2.encoder_attn.o_proj
model.transformer_module.decoder.layers.2.encoder_attn_layer_norm
model.transformer_module.decoder.layers.2.mlp
model.transformer_module.decoder.layers.2.mlp.fc1
model.transformer_module.decoder.layers.2.mlp.fc2
model.transformer_module.decoder.layers.2.mlp.activation_fn
model.transformer_module.decoder.layers.2.final_layer_norm
model.transformer_module.decoder.layers.3
model.transformer_module.decoder.layers.3.self_attn
model.transformer_module.decoder.layers.3.self_attn.k_proj
model.transformer_module.decoder.layers.3.self_attn.v_proj
model.transformer_module.decoder.layers.3.self_attn.q_proj
model.transformer_module.decoder.layers.3.self_attn.o_proj
model.transformer_module.decoder.layers.3.self_attn_layer_norm
model.transformer_module.decoder.layers.3.encoder_attn
model.transformer_module.decoder.layers.3.encoder_attn.k_proj
model.transformer_module.decoder.layers.3.encoder_attn.v_proj
model.transformer_module.decoder.layers.3.encoder_attn.q_proj
model.transformer_module.decoder.layers.3.encoder_attn.o_proj
model.transformer_module.decoder.layers.3.encoder_attn_layer_norm
model.transformer_module.decoder.layers.3.mlp
model.transformer_module.decoder.layers.3.mlp.fc1
model.transformer_module.decoder.layers.3.mlp.fc2
model.transformer_module.decoder.layers.3.mlp.activation_fn
model.transformer_module.decoder.layers.3.final_layer_norm
model.transformer_module.decoder.layers.4
model.transformer_module.decoder.layers.4.self_attn
model.transformer_module.decoder.layers.4.self_attn.k_proj
model.transformer_module.decoder.layers.4.self_attn.v_proj
model.transformer_module.decoder.layers.4.self_attn.q_proj
```

```
model.transformer_module.decoder.layers.4.self_attn.o_proj
model.transformer_module.decoder.layers.4.self_attn_layer_norm
model.transformer_module.decoder.layers.4.encoder_attn
model.transformer_module.decoder.layers.4.encoder_attn.k_proj
model.transformer_module.decoder.layers.4.encoder_attn.v_proj
model.transformer_module.decoder.layers.4.encoder_attn.q_proj
model.transformer_module.decoder.layers.4.encoder_attn.o_proj
model.transformer_module.decoder.layers.4.encoder_attn_layer_norm
model.transformer_module.decoder.layers.4.mlp
model.transformer_module.decoder.layers.4.mlp.fc1
model.transformer_module.decoder.layers.4.mlp.fc2
model.transformer_module.decoder.layers.4.mlp.activation_fn
model.transformer_module.decoder.layers.4.final_layer_norm
model.transformer_module.decoder.layers.5
model.transformer_module.decoder.layers.5.self_attn
model.transformer_module.decoder.layers.5.self_attn.k_proj
model.transformer_module.decoder.layers.5.self_attn.v_proj
model.transformer_module.decoder.layers.5.self_attn.q_proj
model.transformer_module.decoder.layers.5.self_attn.o_proj
model.transformer_module.decoder.layers.5.self_attn_layer_norm
model.transformer_module.decoder.layers.5.encoder_attn
model.transformer_module.decoder.layers.5.encoder_attn.k_proj
model.transformer_module.decoder.layers.5.encoder_attn.v_proj
model.transformer_module.decoder.layers.5.encoder_attn.q_proj
model.transformer_module.decoder.layers.5.encoder_attn.o_proj
model.transformer_module.decoder.layers.5.encoder_attn_layer_norm
model.transformer_module.decoder.layers.5.mlp
model.transformer_module.decoder.layers.5.mlp.fc1
model.transformer_module.decoder.layers.5.mlp.fc2
model.transformer_module.decoder.layers.5.mlp.activation_fn
model.transformer_module.decoder.layers.5.final_layer_norm
model.transformer_module.decoder.layernorm
class_predictor
mask_embedder
mask_embedder.0
mask_embedder.0.0
mask_embedder.0.1
mask_embedder.1
mask_embedder.1.0
mask_embedder.1.1
mask_embedder.2
mask_embedder.2.0
```

mask_embedder.2.1
matcher
criterion

TODO 13: extract the pixel-level module from maskformer and also replace the last layer (mask prediction) with a new convolutional layer (outchannel=1, kernel_size=3, stride=1, padding=1).

```
In [7]: class SegmentModel(nn.Module):
    def __init__(self, model_name):
        super().__init__()
        maskformer = MaskFormerForInstanceSegmentation.from_pretrained(model_name)
        # TODO 13.1: extract pixel-level module and create new mask projection
        self.base_model = maskformer.model.pixel_level_module
        self.activation = {}

        self.base_model.decoder.mask_projection = nn.Conv2d(256, 1, 3, 1, 1)
        def get_activation(name):
            def hook(model, input, output):
                self.activation[name] = output#.detach() # in case that you want to stop the gradient to f
            return hook

        self.base_model.decoder.mask_projection.register_forward_hook(get_activation('mask_projection'))

    def forward(self, pixel_values):
        # TODO 13.2: extract pixel-level module and create new mask projection
        # Note: Do not forget to interpolate the output to have the save size as pixel_values (the input)
        # we will use the bilinear interpolation with align_corners set to False.
        output = self.base_model(pixel_values)
        output = self.activation['mask_projection']
        output = F.interpolate(output, size=pixel_values.shape[-2:], mode='bilinear', align_corners=False)
        return output
```

The number of parameters in the extracted pixel-level module should be around 31,239,099.

```
In [8]: from torchinfo import summary
model = SegmentModel(model_name)

pytorch_total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)

summary(model, input_size=[(32, 3, 32, 32)], dtypes=[torch.float])
```

Loading weights: 0% | 0/432 [00:00<?, ?it/s]

```

Out[8]: =====
=====
Layer (type:depth-idx)                                Output Shape
Param #
=====
SegmentModel                                           [32, 1, 32, 32]
--
└─MaskFormerPixelLevelModule: 1-1                      --
--
    └─MaskFormerSwinBackbone: 2-1                      [32, 96, 8, 8]
--
        └─MaskFormerSwinModel: 3-1                      --
27,519,354
        └─ModuleList: 3-2                              --
2,880
            └─MaskFormerPixelDecoder: 2-2              --
--
                └─MaskFormerFPNModel: 3-3              [32, 256, 2, 2]
3,714,560
                └─Conv2d: 3-4                          [32, 1, 8, 8]
2,305
=====
=====
Total params: 31,239,099
Trainable params: 31,239,099
Non-trainable params: 0
Total mult-adds (Units.GIGABYTES): 2.63
=====
=====
Input size (MB): 0.39
Forward/backward pass size (MB): 335.36
Params size (MB): 124.86
Estimated Total Size (MB): 460.62
=====
=====

```

Insert LoRA into the MaskFormer model

The concept of LoRA is that we are going to estimate the gradient (adaptation matrix) with two smaller matrices (A and B):

$$\text{Adaptation Matrix} = B \times A$$

where $\text{Adaptation Matrix} \in \mathbb{R}^{m \times n}$, $A \in \mathbb{R}^{r \times n}$, and $B \in \mathbb{R}^{m \times r}$. We could make this approximation based on the assumption that Adaptation Matrix has a rank of r . Therefore, the fine-tuned weight becomes

$$\begin{aligned} W &= W_0 + \Delta W \\ &= W_0 + \frac{\alpha}{r} BA \end{aligned}$$

where W denotes the fine-tuned weight, W_0 represents pre-trained weight, ΔW is the gradient and α can be seen as a learning rate. A is initialized using a common initialization, like Kaiming initialization, during the initialization process. On the other hand, B is set to 0 such that the model's output remains the same after injecting LoRA, resulting in a stabilized training process.

To summarize, when injecting LoRA to a layer, we insert new parameters called matrix A and B and initialize them using the above description. Then, we modify the forward pass with `forward_hook` such that the output becomes

$$h = Wx + \frac{\alpha}{r} BAx$$

where x and h are the input and output, respectively. We recommend you read this [blog](#) to learn more about `forward_hook`.

LoRA on Linear Layer

TODO 14: initialize A and B to ones (every entry in the matrix is one), such that we can verify your forward pass after attach hook.

TODO 15: implement the forward hook such that new output h is

$$h = Wx + \frac{\alpha}{r} BAx$$

LoRA on Convolutional Layer

TODO 16: initialize A and B to ones (every entry in the matrix is one), such that we can verify your forward pass after attach hook.

TODO 17: implement the forward hook such that new output h is

$$h = Wx + \frac{\alpha}{r} BAx$$

Note: When injecting LoRA into a convolutional layer with kernel size k , the shape of matrix A and B becomes $(r \times k, \text{in features} \times k)$ and $(\text{out features} \times k, r \times k)$, respectively. It comes from the fact that we can see the weight of the convolutional layer as the weight of $k \times k$ linear layers.

Hint: When you want to declare and initialize a parameter, you can use `torch.nn.Parameter` and `torch.nn.init`, respectively.

```
In [9]: import math
# Initialize LoRA and attach a hook.
def attach_lora(layer, r, lora_alpha, in_features, out_features):
    assert r > 0, "rank must greater than 0."
    # TODO 14: Declare A and B matrices and initialize A and B to ones.
    layer.lora_A = nn.Parameter(torch.ones(r, in_features))
    layer.lora_B = nn.Parameter(torch.ones(out_features, r))

    def hook(model, input, output):
        assert len(input) == 1, "The length of the input must be 1."
        # TODO 15: Compute adaptation matrix (BA) and modify the forward pass.
        x = input[0]
        after_A = x @ layer.lora_A.t()

        after_B = after_A @ layer.lora_B.t()

        adaptation = after_B * (lora_alpha / r)
        return output + adaptation

    return hook

def attach_conv_lora(layer, r, lora_alpha, in_features, out_features, kernel_size):
    assert r > 0, "rank must greater than 0."
    # TODO 16: initialize matrix A and B in LoRA
    layer.lora_A = nn.Parameter(torch.ones(r * kernel_size, in_features * kernel_size))
    layer.lora_B = nn.Parameter(torch.ones(out_features * kernel_size, r * kernel_size))

    def hook(model, input, output):
        assert len(input) == 1, "The length of the input must be 1."
        # TODO 17: Compute adaptation matrix (BA) and modify the forward pass.
        x = input[0]
        delta_W = (layer.lora_B @ layer.lora_A).view(out_features, in_features, kernel_size, kernel_size)
```

```

        adaptation = F.conv2d(
            x, delta_W, stride=layer.stride, padding=layer.padding
        )
    return output + adaptation * (lora_alpha / r)
return hook

```

To test your `forward_hook`, we will check the difference of the output before and after injecting the LoRA when you initialize matrices A and B with ones.

```

In [10]: class DummyLinear(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(10, 20)

    def forward(self, x):
        return self.linear(x)

r, lora_alpha = 1, 4
input_ = torch.arange(10, dtype=torch.float32).unsqueeze(0)
dummy_linear = DummyLinear()
output_before = dummy_linear(input_)
for name, module in dummy_linear.named_modules():
    if isinstance(module, torch.nn.modules.Linear):
        out_features, in_features = module.weight.shape
        h = module.register_forward_hook(attach_lora(module, r, lora_alpha, in_features, out_features))
output_after = dummy_linear(input_)

if torch.all(torch.isclose(output_after - output_before, lora_alpha * input_.sum() * torch.ones_like(output_after))):
    print("Your forward hook seems to be correct.")
else:
    print("There is something wrong with your forward hook.")

```

Your forward hook seems to be correct.

Analyze Low-rank Adaptation (TODO)

How many parameters in DummyLinear module before and after injecting LoRA?

Ans. Before 220 After 250.

Check LoRA in convolutional layer

```
In [11]: class DummyConvolution(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(10, 20, kernel_size=3, stride=3)

    def forward(self, x):
        return self.conv(x)

r, lora_alpha = 1, 4
input_ = torch.ones(1, 10, 9, 9) # torch.arange(10, dtype=torch.float32).unsqueeze(0)
dummy_conv = DummyConvolution()
output_before = dummy_conv(input_)
for name, module in dummy_conv.named_modules():
    if isinstance(module, torch.nn.modules.Conv2d):
        out_features, in_features, kernel_size, _ = module.weight.shape
        assert kernel_size == _
        h = module.register_forward_hook(attach_conv_lora(module, r, lora_alpha, in_features, out_features, l
output_after = dummy_conv(input_)

if torch.all(torch.isclose(output_after - output_before, 1080 * torch.ones_like(output_before))):
    print("Your forward hook seems to be correct.")
else:
    print("There is something wrong with your forward hook.")
```

Your forward hook seems to be correct.

Instruction

TODO 18-19: Change the initialization of A and B where A is initialized with Kaiming Uniform ($a = \sqrt{5}$), and B is set to 0.

```
In [12]: import math
# Initialize LoRA and attach a hook.
def attach_lora(layer, r, lora_alpha, in_features, out_features):
    assert r > 0, "rank must greater than 0."
    # TODO 18: initialize A with kaiming uniform with a = sqrt(5) and initialize B to 0.
    layer.lora_A = nn.Parameter(torch.empty(r, in_features))
    layer.lora_B = nn.Parameter(torch.zeros(out_features, r))

    nn.init.kaiming_uniform_(layer.lora_A, a=math.sqrt(5))
```

```

def hook(model, input, output):
    assert len(input) == 1, "The length of the input must be 1."
    # Copy from TODO 15
    x = input[0]
    after_A = x @ layer.lora_A.t()

    after_B = after_A @ layer.lora_B.t()

    adaptation = after_B * (lora_alpha / r)
    return output + adaptation

return hook

def attach_conv_lora(layer, r, lora_alpha, in_features, out_features, kernel_size):
    assert r > 0, "rank must greater than 0."
    # TODO 19: initialize A with kaiming uniform with a = sqrt(5) and initialize B to 0
    layer.lora_A = nn.Parameter(torch.empty(r * kernel_size, in_features * kernel_size))
    layer.lora_B = nn.Parameter(torch.zeros(out_features * kernel_size, r * kernel_size))

    nn.init.kaiming_uniform_(layer.lora_A, a=math.sqrt(5))

def hook(model, input, output):
    assert len(input) == 1, "The length of the input must be 1."
    # COPY from TODO 17
    x = input[0]
    delta_W = (layer.lora_B @ layer.lora_A).view(out_features, in_features, kernel_size, kernel_size)

    adaptation = F.conv2d(
        x, delta_W, stride=layer.stride, padding=layer.padding
    )
    return output + adaptation * (lora_alpha / r)
return hook

```

We only inject LoRA into the convolutional layer in the decoder block.

TODO 20: inject lora into a decoder block

```

In [13]: r, lora_alpha = 1, 4
def attach_lora_to_maskformer(model, r, lora_alpha):
    hooks = []
    for name, module in model.named_modules():

```

```

# TODO 20: inject lora into convolutional layers in the decoder block
# Do not forget to append registered hook to hooks
if isinstance(module, torch.nn.Conv2d) and 'decoder' in name:
    out_f, in_f, k, _ = module.weight.shape
    h = module.register_forward_hook(
        attach_conv_lora(module, r, lora_alpha, in_f, out_f, k)
    )
    hooks.append(h)
return hooks

hooks = attach_lora_to_maskformer(model, r, lora_alpha)

```

We freeze every layer except LoRA layer (A and B matrix), bias in the decoder, and the last layer (segment_prediction).

TODO 21: freeze every layer except LoRA layer (A and B matrix), bias in the decoder, and the last layer (segment_prediction).

```

In [14]: for n, p in model.named_parameters():
# TODO 21: freeze every layer except LoRA layer, bias in the decoder and the last layer.
if 'lora' in n or ('decoder' in n and 'bias' in n) or 'segmentation_prediction' in n:
    p.requires_grad = True
else:
    p.requires_grad = False

```

The number of learnable parameters is around 28k.

```

In [15]: pytorch_total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(pytorch_total_params)

```

28586

Training

```

In [16]: from tqdm import tqdm

# Save on the trainable parameters
def save_trainable_params(model, path):
    trainable_state_dict = {}
    for n, p in model.named_parameters():
        if p.requires_grad:
            trainable_state_dict[n] = p

```

```
torch.save(trainable_state_dict, path)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
criteria = nn.BCEWithLogitsLoss()
# Initialize Adam optimizer
optimizer = torch.optim.Adam(model.parameters(), lr=3e-4)
# Set number of epochs and batch size
num_epochs = 10
training_losses = []
validation_losses = []
for epoch in range(num_epochs):
    print(f"Epoch {epoch} | Training")
    # Set model in training mode
    model.train()
    train_loss, val_loss = [], []
    # Training loop
    for idx, batch in enumerate(tqdm(train_dataloader)):
        # Reset the parameter gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model(batch["pixel_values"].to(device))
        # Backward propagation
        loss = criteria(outputs.squeeze(1), batch["segmentation"].to(device))
        train_loss.append(loss.item())
        loss.backward()
        if idx % 100 == 0:
            print(" Training loss: ", round(sum(train_loss)/len(train_loss), 6))
        # Optimization
        optimizer.step()
        torch.cuda.empty_cache()
    # Average train epoch loss
    train_loss = sum(train_loss)/len(train_loss)
    # Set model in evaluation mode
    model.eval()
    start_idx = 0
    print(f"Epoch {epoch} | Validation")
    for idx, batch in enumerate(tqdm(val_dataloader)):
        with torch.no_grad():
```

```

    # Forward pass
    outputs = model(batch["pixel_values"].to(device))
    # Get validation loss
    loss = criteria(outputs.squeeze(1), batch["segmentation"].to(device))
    val_loss.append(loss.item())
    if idx % 50 == 0:
        print(" Validation loss: ", round(sum(val_loss)/len(val_loss), 6))
    torch.cuda.empty_cache()
    # Average validation epoch loss
    val_loss = sum(val_loss)/len(val_loss)

    training_losses.append(train_loss)
    validation_losses.append(val_loss)
    # Print epoch losses
    print(f"Epoch {epoch} | train_loss: {train_loss} | validation_loss: {val_loss}")

save_trainable_params(model, "./lora_weight.ckpt")

```

Epoch 0 | Training

0%| | 1/212 [00:00<01:39, 2.12it/s]

Training loss: 0.675568

48%| | 101/212 [00:34<00:34, 3.26it/s]

Training loss: 0.402911

95%| | 201/212 [01:07<00:04, 2.27it/s]

Training loss: 0.336924

100%| | 212/212 [01:11<00:00, 2.97it/s]

Epoch 0 | Validation

2%|| | 1/61 [00:00<00:17, 3.41it/s]

Validation loss: 0.267857

84%| | 51/61 [00:16<00:02, 3.65it/s]

Validation loss: 0.245162

100%| | 61/61 [00:18<00:00, 3.25it/s]

Epoch 0 | train_loss: 0.3327035981950895 | validation_loss: 0.24224924186214072

Epoch 1 | Training

0%| | 1/212 [00:00<01:01, 3.44it/s]

Training loss: 0.291073

48%| | 101/212 [00:33<00:34, 3.20it/s]

Training loss: 0.235379

95%| | 201/212 [01:05<00:03, 3.44it/s]

```
Training loss: 0.223814
100%|██████████| 212/212 [01:08<00:00, 3.09it/s]
Epoch 1 | Validation
 2%||          | 1/61 [00:00<00:18, 3.23it/s]
Validation loss: 0.225113
84%|██████████| 51/61 [00:16<00:02, 3.68it/s]
Validation loss: 0.210424
100%|██████████| 61/61 [00:18<00:00, 3.26it/s]
Epoch 1 | train_loss: 0.2232365057153522 | validation_loss: 0.20803217028008134
Epoch 2 | Training
 0%|          | 1/212 [00:00<01:04, 3.28it/s]
Training loss: 0.252178
48%|██████████| 101/212 [00:32<00:33, 3.28it/s]
Training loss: 0.213039
95%|██████████| 201/212 [01:05<00:03, 3.38it/s]
Training loss: 0.206952
100%|██████████| 212/212 [01:08<00:00, 3.09it/s]
Epoch 2 | Validation
 2%||          | 1/61 [00:00<00:17, 3.36it/s]
Validation loss: 0.219254
84%|██████████| 51/61 [00:16<00:02, 3.51it/s]
Validation loss: 0.2008
100%|██████████| 61/61 [00:19<00:00, 3.20it/s]
Epoch 2 | train_loss: 0.20566173642873764 | validation_loss: 0.19836096856437746
Epoch 3 | Training
 0%|          | 1/212 [00:00<01:03, 3.34it/s]
Training loss: 0.168585
48%|██████████| 101/212 [00:31<00:38, 2.90it/s]
Training loss: 0.198685
95%|██████████| 201/212 [01:04<00:03, 2.92it/s]
Training loss: 0.197185
100%|██████████| 212/212 [01:08<00:00, 3.11it/s]
Epoch 3 | Validation
 2%||          | 1/61 [00:00<00:17, 3.46it/s]
Validation loss: 0.21177
84%|██████████| 51/61 [00:15<00:02, 3.63it/s]
Validation loss: 0.193943
```

```
100%|██████████| 61/61 [00:18<00:00, 3.30it/s]
Epoch 3 | train_loss: 0.19712737666548424 | validation_loss: 0.19151166301281725
Epoch 4 | Training
  0%|          | 1/212 [00:00<01:01, 3.45it/s]
    Training loss: 0.168824
 48%|██████    | 101/212 [00:32<00:34, 3.25it/s]
    Training loss: 0.192582
 95%|██████████| 201/212 [01:05<00:03, 2.84it/s]
    Training loss: 0.191559
100%|██████████| 212/212 [01:08<00:00, 3.10it/s]
Epoch 4 | Validation
  2%||         | 1/61 [00:00<00:20, 2.86it/s]
    Validation loss: 0.207334
 84%|██████████| 51/61 [00:16<00:02, 3.60it/s]
    Validation loss: 0.190124
100%|██████████| 61/61 [00:18<00:00, 3.24it/s]
Epoch 4 | train_loss: 0.19289342749793575 | validation_loss: 0.18776327640306753
Epoch 5 | Training
  0%|          | 1/212 [00:00<01:00, 3.48it/s]
    Training loss: 0.217718
 48%|██████    | 101/212 [00:33<00:32, 3.37it/s]
    Training loss: 0.193613
 95%|██████████| 201/212 [01:05<00:03, 3.09it/s]
    Training loss: 0.189196
100%|██████████| 212/212 [01:08<00:00, 3.10it/s]
Epoch 5 | Validation
  2%||         | 1/61 [00:00<00:18, 3.18it/s]
    Validation loss: 0.208689
 84%|██████████| 51/61 [00:16<00:02, 3.41it/s]
    Validation loss: 0.187809
100%|██████████| 61/61 [00:19<00:00, 3.12it/s]
Epoch 5 | train_loss: 0.18850917260180106 | validation_loss: 0.18538201783524186
Epoch 6 | Training
  0%|          | 1/212 [00:00<01:11, 2.97it/s]
    Training loss: 0.207962
 48%|██████    | 101/212 [00:33<00:38, 2.88it/s]
    Training loss: 0.184937
```

```
95%|██████████ | 201/212 [01:05<00:04, 2.74it/s]
Training loss: 0.185343
100%|██████████ | 212/212 [01:09<00:00, 3.07it/s]
Epoch 6 | Validation
2%|| | 1/61 [00:00<00:17, 3.35it/s]
Validation loss: 0.202083
84%|██████████ | 51/61 [00:15<00:02, 3.68it/s]
Validation loss: 0.183579
100%|██████████ | 61/61 [00:18<00:00, 3.29it/s]
Epoch 6 | train_loss: 0.18493433915219218 | validation_loss: 0.18132584187828127
Epoch 7 | Training
0%| | 1/212 [00:00<00:56, 3.72it/s]
Training loss: 0.124281
48%|██████ | 101/212 [00:32<00:31, 3.50it/s]
Training loss: 0.183088
95%|██████████ | 201/212 [01:05<00:04, 2.59it/s]
Training loss: 0.182769
100%|██████████ | 212/212 [01:08<00:00, 3.10it/s]
Epoch 7 | Validation
2%|| | 1/61 [00:00<00:16, 3.55it/s]
Validation loss: 0.198601
84%|██████████ | 51/61 [00:16<00:02, 3.59it/s]
Validation loss: 0.182597
100%|██████████ | 61/61 [00:19<00:00, 3.19it/s]
Epoch 7 | train_loss: 0.1824559848052713 | validation_loss: 0.1805749071426079
Epoch 8 | Training
0%| | 1/212 [00:00<01:03, 3.31it/s]
Training loss: 0.173099
48%|██████ | 101/212 [00:32<00:32, 3.39it/s]
Training loss: 0.179265
95%|██████████ | 201/212 [01:05<00:03, 3.21it/s]
Training loss: 0.180441
100%|██████████ | 212/212 [01:08<00:00, 3.09it/s]
Epoch 8 | Validation
2%|| | 1/61 [00:00<00:17, 3.53it/s]
Validation loss: 0.198054
84%|██████████ | 51/61 [00:15<00:02, 3.54it/s]
```

Validation loss: 0.184222

100%|██████████| 61/61 [00:18<00:00, 3.29it/s]

Epoch 8 | train_loss: 0.1803541101234139 | validation_loss: 0.18232496400348475

Epoch 9 | Training

0%| | 1/212 [00:00<01:02, 3.35it/s]

Training loss: 0.175073

48%|██████ | 101/212 [00:32<00:31, 3.58it/s]

Training loss: 0.180328

95%|██████████| 201/212 [01:05<00:04, 2.58it/s]

Training loss: 0.179212

100%|██████████| 212/212 [01:08<00:00, 3.09it/s]

Epoch 9 | Validation

2%|| | 1/61 [00:00<00:17, 3.33it/s]

Validation loss: 0.197912

84%|██████████| 51/61 [00:15<00:02, 3.59it/s]

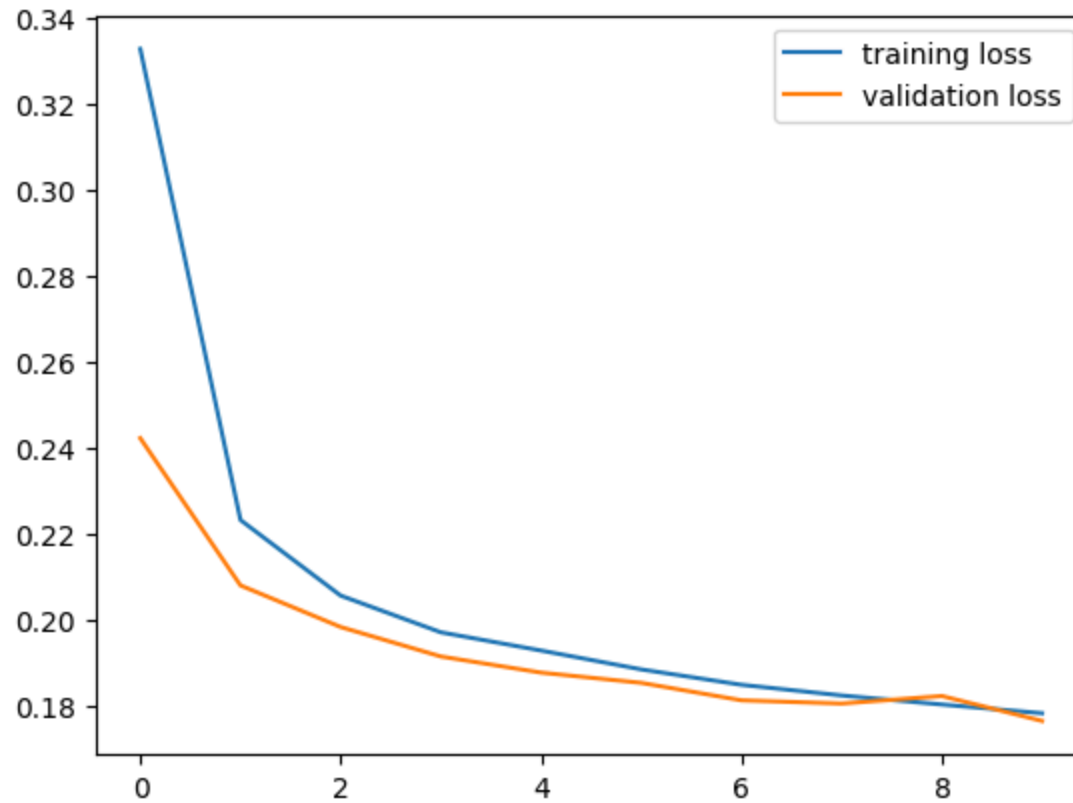
Validation loss: 0.178956

100%|██████████| 61/61 [00:18<00:00, 3.29it/s]

Epoch 9 | train_loss: 0.17832594178617 | validation_loss: 0.176533720776683

Plot training and validation loss.

```
In [17]: plt.plot(range(num_epochs), training_losses, label="training loss")
plt.plot(range(num_epochs), validation_losses, label="validation loss")
plt.legend()
plt.show()
```



```
In [18]: test_loss = []
         for idx, batch in enumerate(tqdm(test_dataloader)):
             with torch.no_grad():
                 outputs = model(batch["pixel_values"].to(device))
                 loss = criteria(outputs.squeeze(1), batch["segmentation"].to(device))
                 test_loss.append(loss.item())
             torch.cuda.empty_cache()
         # Average validation epoch loss
         test_loss = sum(test_loss)/len(test_loss)
         print("Test Loss:", test_loss)
```

```
100%|██████████| 31/31 [00:09<00:00, 3.20it/s]
Test Loss: 0.17830580665219214
```

Inference

After fine-tuning the model, we need to load the model back for inference. First, we load the pre-trained weight. Next, we attach LoRA to the foundation model and then load the LoRA's weight.

```
In [19]: model = SegmentModel(model_name)
hooks = attach_lora_to_maskformer(model, r, lora_alpha)

model.load_state_dict(torch.load("./lora_weight.ckpt"), strict=False)
model.cuda()

test_loss = []
for idx, batch in enumerate(tqdm(test_dataloader)):
    with torch.no_grad():
        outputs = model(batch["pixel_values"].to(device))
        loss = criteria(outputs.squeeze(1), batch["segmentation"].to(device))
        test_loss.append(loss.item())
    torch.cuda.empty_cache()
# Average validation epoch loss
test_loss = sum(test_loss)/len(test_loss)
print("Test Loss:", test_loss)
```

```
Loading weights: 0%|          | 0/432 [00:00<?, ?it/s]
100%|██████████| 31/31 [00:09<00:00, 3.24it/s]
Test Loss: 0.49714416169351144
```

```
In [20]: test_dataloader = DataLoader(
    test_dataset,
    batch_size=1,
    shuffle=False,
    collate_fn=collate_fn
)
num_images = 10
model.cuda()
for idx, batch in enumerate(test_dataloader):
    image = unnormalize_image(batch["pixel_values"][0])
    outputs = model(batch["pixel_values"].to(device))
    outputs = F.sigmoid(outputs).cpu().detach().numpy()

    mask = batch["segmentation"][0]

    fig, axs = plt.subplots(ncols=3, figsize=(20, 10))
```

```

    axs[0].imshow(np.array(rearrange(image, "c h w -> h w c")))
    axs[1].imshow(outputs[0].squeeze())
    axs[2].imshow(np.array(mask))
    axs[0].set_axis_off()
    axs[1].set_axis_off()
    axs[2].set_axis_off()
    plt.show()

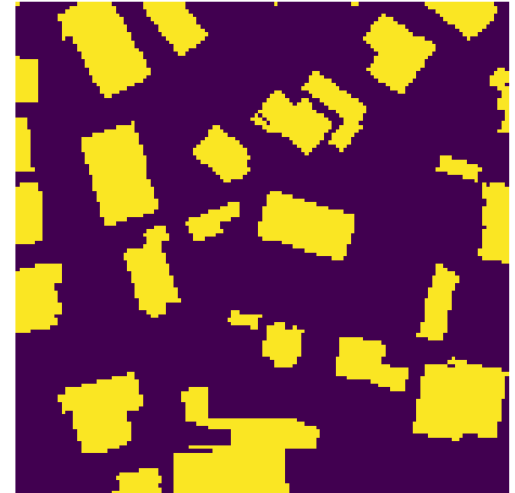
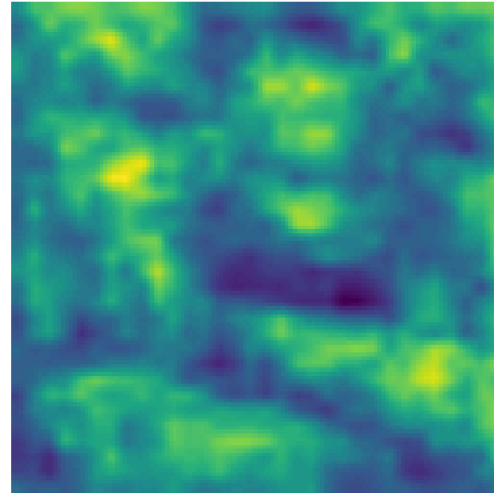
    if idx >= num_images:
        break

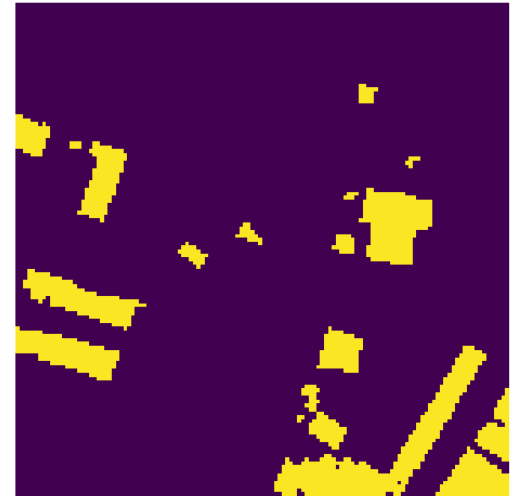
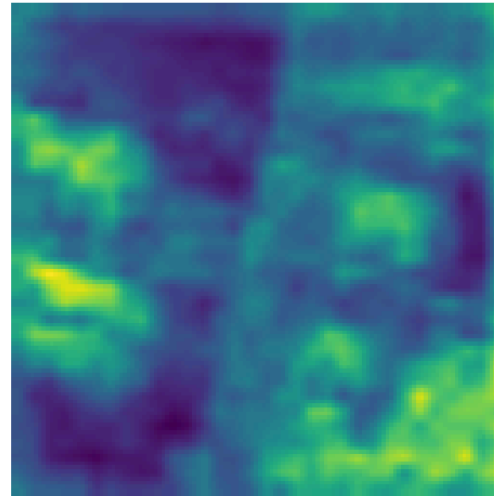
```

```

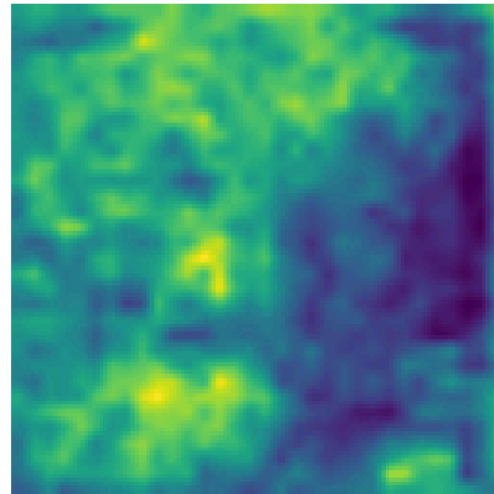
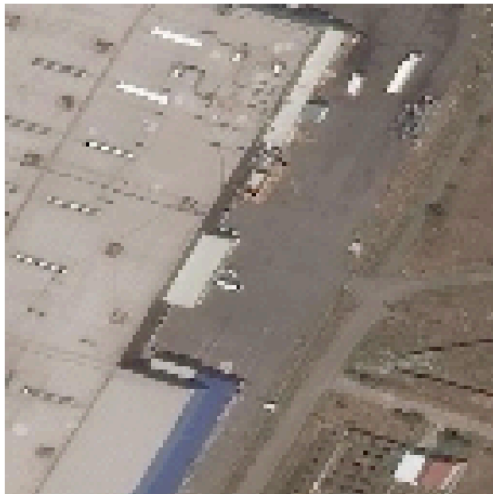
/tmp/ipykernel_180391/1125473636.py:16: DeprecationWarning: __array_wrap__ must accept context and return_s
calar arguments (positionally) in the future. (Deprecated NumPy 2.0)
    image = image * np.array([0.229, 0.224, 0.225])[:, None, None]
/tmp/ipykernel_180391/1125473636.py:17: DeprecationWarning: __array_wrap__ must accept context and return_s
calar arguments (positionally) in the future. (Deprecated NumPy 2.0)
    image = image + np.array([0.485, 0.456, 0.406])[:, None, None]
/tmp/ipykernel_180391/1214426544.py:17: DeprecationWarning: __array__ implementation doesn't accept a copy
keyword, so passing copy=False failed. __array__ must implement 'dtype' and 'copy' keyword arguments. To le
arn more, see the migration guide https://numpy.org/devdocs/numpy\_2\_0\_migration\_guide.html#adapting-to-changes-in-the-copy-keyword
    axs[0].imshow(np.array(rearrange(image, "c h w -> h w c")))
/tmp/ipykernel_180391/1214426544.py:19: DeprecationWarning: __array__ implementation doesn't accept a copy
keyword, so passing copy=False failed. __array__ must implement 'dtype' and 'copy' keyword arguments. To le
arn more, see the migration guide https://numpy.org/devdocs/numpy\_2\_0\_migration\_guide.html#adapting-to-changes-in-the-copy-keyword
    axs[2].imshow(np.array(mask))

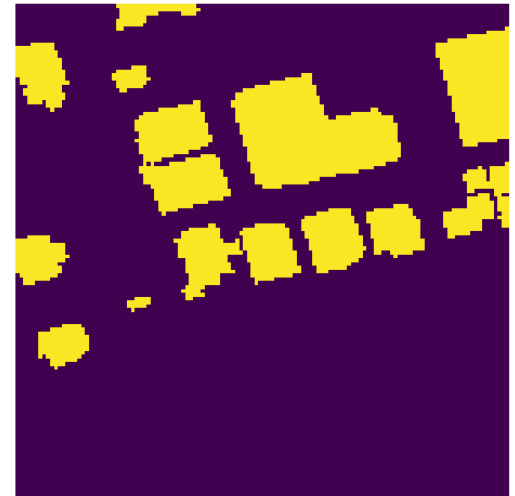
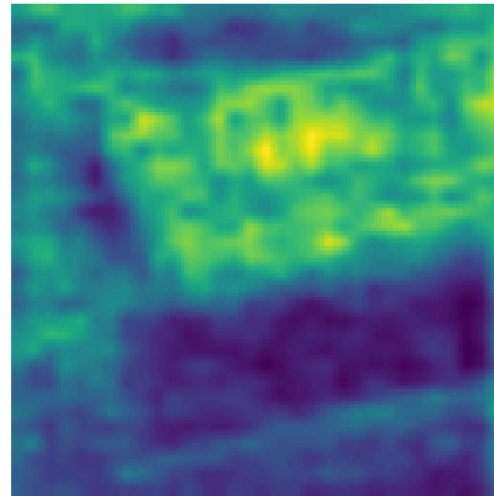
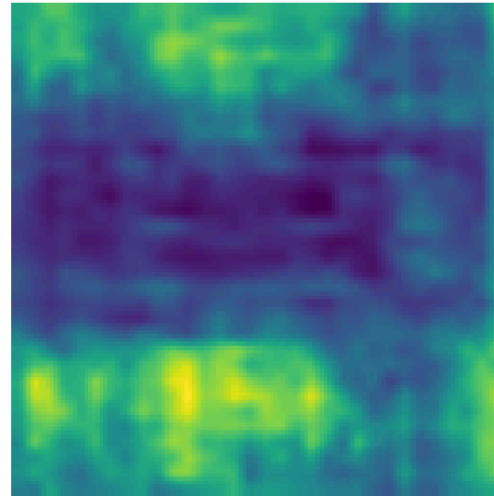
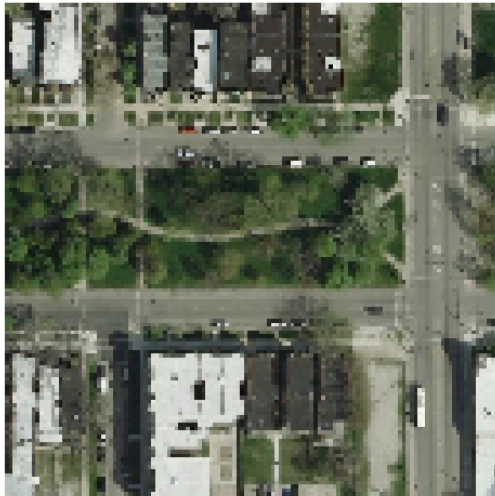
```

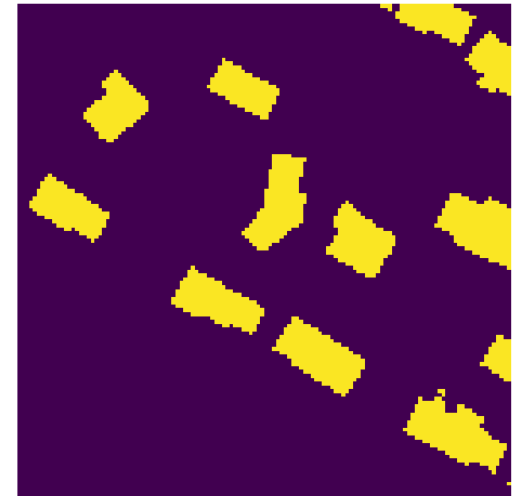
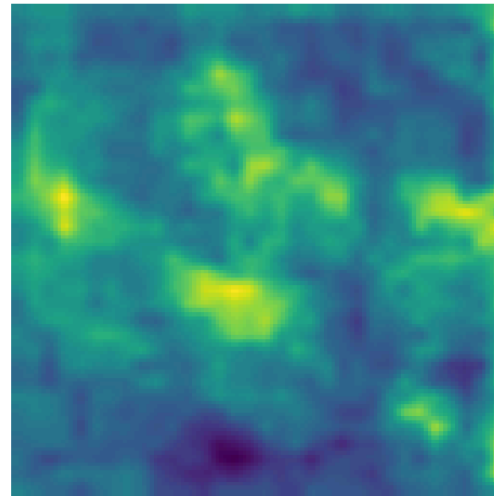
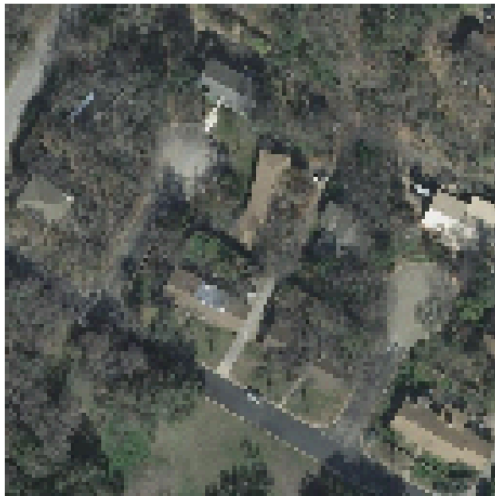
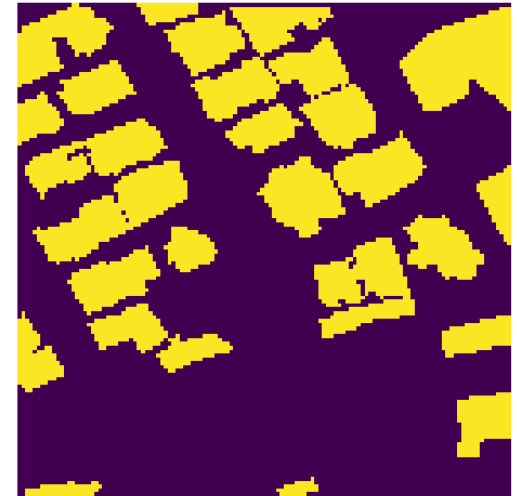
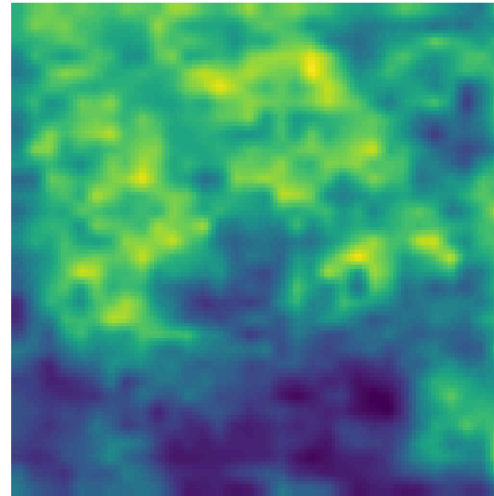




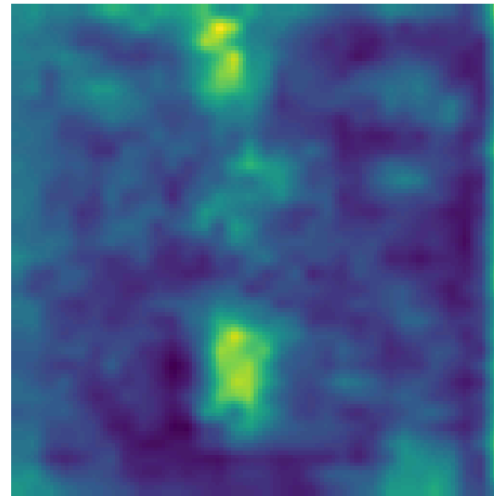
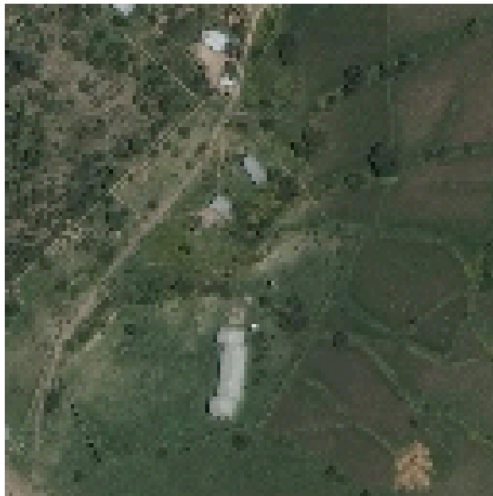
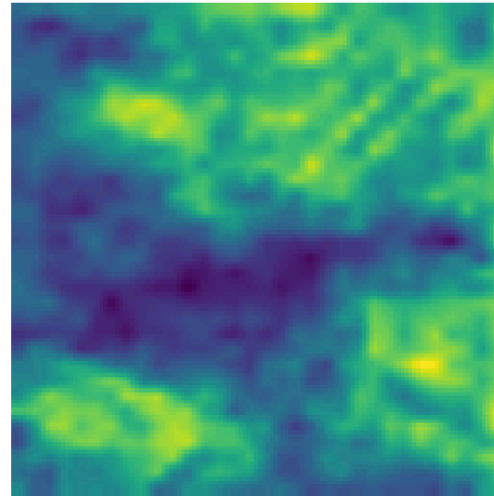
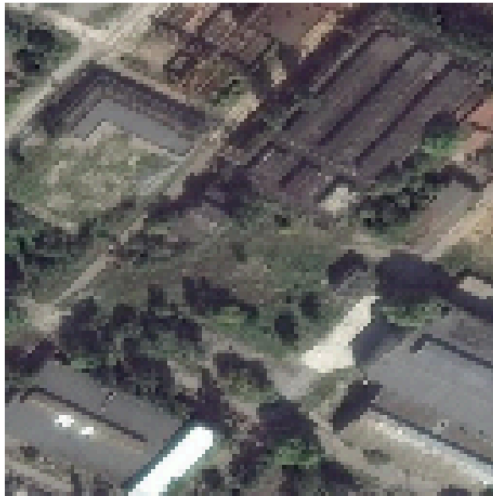
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.01960783576965336..1.0000000076293944].



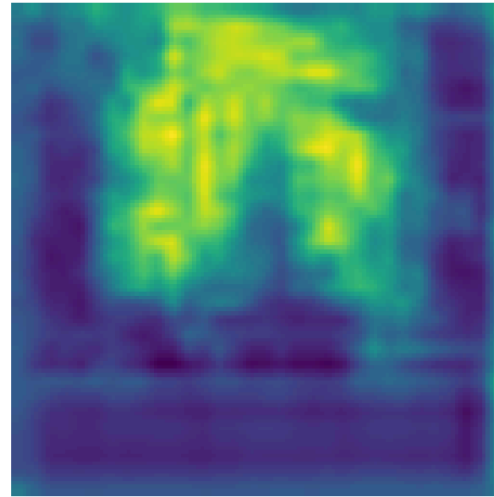
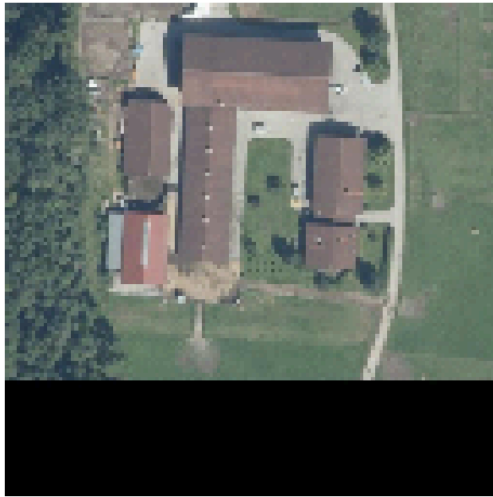




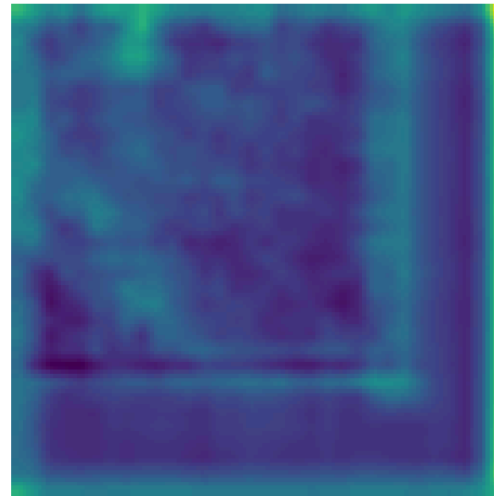
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.05098038780689235..1.0000000236034394].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-2.288818357065736e-08..0.9921568689346314].



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-2.288818357065736e-08..0.63137255859375].



(Optional) Part 3: Low-rank Adaptation on Diffusion

In this section, we combine sections 1 and 2. Instead of controlling the diffusion with digits, we will regulate it with text by using CLIP, trained to align the text and image embedding in the same space.

To make it even more interesting, we fill color into the MNIST dataset and use the prompt to generate the desired images. In addition, we color only digits 0 to 4 while leaving digits 5-9 to their original color (white).

Preprocessing

First, we create Dataset and declare six colors, including red, green, blue, purple, yellow and white. Then we define the corresponding prompt for each image.

```
In [21]: import random
from PIL import Image

COLOR = {
    "red": (1, 0, 0),
    "green": (0, 1, 0),
    "blue": (0, 0, 1),
    "purple": (1, 0, 1),
    "yellow": (1, 1, 0),
    "white": (1, 1, 1)
}

def fill_color(img, color):
    """ Filled color into the image.

    Args:
        img (PIL.Image): A pillow image to be filled.
        color (str): A color to be used.
    Returns:
        The modified image (PIL.Image)
    """
    r, g, b = COLOR[color]
    data = np.array(img)  # "data" is a height x width x 4 numpy array
    data[..., 0] *= r
    data[..., 1] *= g
    data[..., 2] *= b
    data = data.astype(np.uint8)
    img = Image.fromarray(data)
    return img

class MNISTCaptionDataset(Dataset):
```

```

def __init__(self, tokenizer, color=False, *args, **kwargs):
    """ Initialize MNIST dataset with text label.

    Args:
        color (bool): If set to true, the digit is filled with random color. Default to False.
    """
    assert kwargs.get("transform") is None, "Do not support transform function."
    self.data = datasets.MNIST(*args, **kwargs)
    self.tokenizer = tokenizer
    self.color = color
    self.classes = list(map(lambda text_label: text_label.split()[-1], self.data.classes))
    # Default transform function
    self.transform = Compose([
        ToTensor(), Resize((32, 32)), normalize_to_neg_one_to_one
    ])

def __len__(self):
    return len(self.data)

def __getitem__(self, idx):
    img, label = self.data[idx]
    img = img.convert('RGB')
    color = ""
    if self.color and label in list(range(0, 5)):
        color = random.choice(list(COLOR.keys()))
        color = "" if color == "white" else color
    if color != "":
        img = fill_color(img, color)
    img = self.transform(img)

    text_label = " ".join([self.classes[label], color]).strip()
    input_ids = self.tokenizer(text_label, padding="max_length", truncation=True, max_length=self.tokenizer.max_length)
    return img, input_ids

```

```

In [22]: from transformers import CLIPTextModel, CLIPTokenizer

mnist_kwargs = {
    "root": "./data",
    # download=True,
}

tokenizer = CLIPTokenizer.from_pretrained("CompVis/stable-diffusion-v1-4", subfolder="tokenizer", revision="main")
text_encoder = CLIPTextModel.from_pretrained("CompVis/stable-diffusion-v1-4", subfolder="text_encoder", revision="main")

```

```

train_data = MNISTCaptionDataset(tokenizer, color=True, train=True, **mnist_kwargs)
test_data = MNISTCaptionDataset(tokenizer, color=True, train=False, **mnist_kwargs)

train_dataloader = DataLoader(train_data, batch_size=128)
test_dataloader = DataLoader(test_data, batch_size=128)

```

```

tokenizer_config.json: 0%|          | 0.00/806 [00:00<?, ?B/s]
vocab.json: 0.00B [00:00, ?B/s]
merges.txt: 0.00B [00:00, ?B/s]
special_tokens_map.json: 0%|          | 0.00/472 [00:00<?, ?B/s]
config.json: 0%|          | 0.00/592 [00:00<?, ?B/s]
text_encoder/model.safetensors: 0%|          | 0.00/492M [00:00<?, ?B/s]
Loading weights: 0%|          | 0/196 [00:00<?, ?it/s]

```

CLIPTextModel LOAD REPORT from: CompVis/stable-diffusion-v1-4

Key	Status		
text_model.embeddings.position_ids	UNEXPECTED		

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

```

-----
NameError                                Traceback (most recent call last)
Cell In[22], line 10
      7 tokenizer = CLIPTokenizer.from_pretrained("CompVis/stable-diffusion-v1-4", subfolder="tokenizer", r
revision=None)
      8 text_encoder = CLIPTextModel.from_pretrained("CompVis/stable-diffusion-v1-4", subfolder="text_encod
er", revision=None)
--> 10 train_data = MNISTCaptionDataset(tokenizer, color=True, train=True, **mnist_kwargs)
      11 test_data = MNISTCaptionDataset(tokenizer, color=True, train=False, **mnist_kwargs)
      13 train_dataloader = DataLoader(train_data, batch_size=128)

Cell In[21], line 39, in MNISTCaptionDataset.__init__(self, tokenizer, color, *args, **kwargs)
     33 """ Initialize MNIST dataset with text label.
     34
     35 Args:
     36     color (bool): If set to true, the digit is filled with random color. Default to False.
     37 """
     38 assert kwargs.get("transform") is None, "Do not support transform function."
--> 39 self.data = datasets.MNIST(*args, **kwargs)
     40 self.tokenizer = tokenizer
     41 self.color = color

NameError: name 'datasets' is not defined

```

```

In [ ]: for i in range(10):
        image, label = train_data[i]
        text = tokenizer.decode(label, skip_special_tokens=True)
        plt.imshow(rearrange(unnormalize_to_zero_to_one(image), "c h w -> h w c"))
        plt.title("Prompt: " + text)
        plt.axis(False)
        plt.show()

```

Load Model

After preprocessing the data, we load the diffusion model and then inject the LoRA.

```

In [ ]: unet = Unet(
        dim = 64,
        dim_mults = (1, 2, 4, 8),
    )

```

```
model = Diffusion(  
    unet,  
    linear_scheduler,  
    resolution=(32, 32),  
)  
  
checkpoint = torch.load("./last_weight.ckpt")  
model.load_state_dict(checkpoint["model_state_dict"])
```

Insert LoRA into diffusion model

Which modules should you inject the LoRA? (OT 1)

Ans.

Instruction

OT 2: inject LoRA into diffusion model

```
In [ ]: r = 8  
hooks = []  
for name, module in model.named_modules():  
    # OT 2: inject LoRA  
    pass
```

After inserting LoRA, we freeze the model and train only LoRA.

Instruction

OT 3: freeze model and train only matrices A and B, and bias.

```
In [ ]: for n, p in model.named_parameters():  
    # OT 3: freeze model and train only matrices A and B, and bias.  
    pass
```

Training

```

In [ ]: class Trainer:
    def __init__(
        self,
        model,
        tokenizer,
        text_encoder,
        train_dataloader,
        val_dataloader,
        ckpt_path,
        resolution=None,
        lr=1e-4,
        device="cuda",
    ):
        self.model = model
        self.tokenizer = tokenizer
        self.text_encoder = text_encoder

        self.train_dataloader = train_dataloader
        self.val_dataloader = val_dataloader
        self.resolution = next(iter(train_dataloader))[0].shape[-2:] if resolution == None else resolution

        self.lr = lr
        self.configure_optimizers()

        self.ckpt_path = ckpt_path
        self.device = device
        self.epoch = 0
        self.mode = "train"

        self.to(device)

    def configure_optimizers(self):
        """Construct Optimizer"""
        self.optimizer = torch.optim.Adam(self.parameters(), lr=self.lr)

    def _step(self, batch):
        """One step forward pass

        Args:
            batch (Tuple[torch.tensor]): A mini-batch data to be processed. The first index must contain image
            and the latter are the corresponded label. The shape of image is (B, C, H, W),

```

```

        while the shape of label is (B).

Returns:
    loss (torch.tensor): A loss value to be used for updating the model.

"""
x, context = batch
# TODO 12: calculate the loss given batch
# Pseudo code:
# 1. acquire the context embedding through self.embedding
# 2. expand dimension such that the shape of embedding is (batch_size, 1, dimension)
# 3. calculate loss using self.model.p_losses
context = self.text_encoder(context, return_dict=False)[0]
losses = self.model.p_losses(x, context)
return losses

def optimizer_step(self, loss):
    """Update model based on the given loss.

    Args:
        loss (torch.tensor): A loss value to be differentiated (Note: must be able to compute a gradient)

    """
    loss.backward()
    self.optimizer.step()
    self.optimizer.zero_grad()

def train_loop(self, epoch):
    """One epoch training loop.

    Args:
        epoch (int): A current epoch.

    Returns:
        train_loss (np.array): A list of loss in each batch.

    """
    self.train()
    with tqdm(self.train_dataloader, unit="batch", leave=False) as tbatch:
        train_loss = []
        for batch in tbatch:
            tbatch.set_description(f"Epoch {epoch+1}")

```

```

        batch = list(
            map(lambda x: x.to(self.device), batch)
        ) # allocate data on the pre-defined device.
        loss = self._step(batch) # forward pass and calculate loss
        self.optimizer_step(loss) # backward pass / updating the parameters
        train_loss.append(loss.item())
        tbatch.set_postfix(loss=loss.item())
        train_loss = np.array(train_loss)
    return train_loss

def val_loop(self, epoch):
    """One epoch validating loop.

    Args:
        epoch (int): A current epoch.

    Returns:
        train_loss (np.array): A list of loss in each batch.

    """
    self.eval()
    # with self.ema.average_parameters(): # Copy EMA weight to model and restore after exiting `with`
    with tqdm(self.val_dataloader, unit="batch", leave=False) as tbatch:
        val_loss = []
        for batch in tbatch:
            tbatch.set_description(f"Epoch {epoch+1}")
            batch = list(
                map(lambda x: x.to(self.device), batch)
            ) # allocate data on the pre-defined device.
            loss = (
                self._step(batch).detach().cpu()
            ) # forward pass and calculate loss
            val_loss.append(loss.item())
            tbatch.set_postfix(loss=loss.item())
        val_loss = np.array(val_loss)
    return val_loss

def fit(self, epochs, num_val_sampler=2, ckpt_path=None, resume=False, sampling_round=10):
    """Train the model

    Args:
        epochs (int): A number of epochs to be trained.

```

```

Returns:
    train_epoch (np.array): A list of training loss in each epoch.
    val_epoch (np.array): A list of validation loss in each epoch.

"""
best_val_loss = None
ckpt_path = self.ckpt_path if ckpt_path is None else ckpt_path
last_path = os.path.join(ckpt_path, "last_weight.ckpt")
print("Save:", last_path)
self.to(self.device)

train_loss_epoch = []
val_loss_epoch = []

epochs = range(self.epoch, self.epoch+epochs) if resume == True else range(epochs)
for epoch in epochs:
    self.epoch = epoch

    train_loss = self.train_loop(epoch) # Training Model
    val_loss = self.val_loop(epoch) # Evaluating Model

    if (epoch+1)%sampling_round == 0 or epoch == 0:
        self.sampling_image(num_val_sampler) # Generating new images

    print(
        f"Epoch {epoch+1}: Train Loss {train_loss.mean().item()}, Val Loss {val_loss.mean().item()}"
    )
    train_loss_epoch.append(train_loss.mean().item())
    val_loss_epoch.append(val_loss.mean().item())

    torch.cuda.empty_cache() # Clear GPU cache

self.save(last_path)
train_loss_epoch = np.array(train_loss_epoch)
val_loss_epoch = np.array(val_loss_epoch)
return train_loss_epoch, val_loss_epoch

def sampling_image(self, num_images):
    """Sampling new images

    The sampling images was exhibited in a column format, using matplotlib.

```

```

    Args:
        num_images (int): A number of images to be generated.

    """
    self.eval()
    fig, axs = plt.subplots(ncols=num_images)
    text_label = random.choices(["zero", "one", "two", "three", "four", "five", "six", "seven", "eight", "nine"], k=num_images)
    text_label = list(map(lambda x: " ".join([x, random.choice(list(COLOR.keys()))]), text_label))
    input_ids = self.tokenizer(text_label, padding="max_length", truncation=True, max_length=self.tokenizer.max_length)
    context = self.text_encoder(input_ids, return_dict=False)[0]
    sampled_images = self.model.sample(batch_size = num_images, context=context)#
    for idx_sampler in range(num_images):
        axs[idx_sampler].imshow(rearrange(unnormalize_to_zero_to_one(sampled_images[idx_sampler]), "c h w"))
        axs[idx_sampler].set_axis_off()
        axs[idx_sampler].set_title(text_label[idx_sampler])
    plt.show()

def save(self, path):
    """ Save trainer state """
    path = self.ckpt_path if path is None else path
    torch.save({
        "epoch": self.epoch,
        "model_state_dict": self.model.state_dict(),
        "optimizer_state_dict": self.optimizer.state_dict(),
        "text_encoder": self.text_encoder.state_dict(),
    }, path)

def load(self, path):
    """ Load trainer state """
    checkpoint = torch.load(path)
    self.epoch = checkpoint["epoch"]
    self.model.load_state_dict(checkpoint["model_state_dict"])
    self.optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
    self.text_encoder.load_state_dict(checkpoint["text_encoder"])

def train(self):
    """ Set trainer to train mode """
    if self.mode != "train":
        self.model.train()
        self.mode = "train"

```

```

def eval(self):
    """ Set trainer to evaluate mode """
    if self.mode != "eval":
        self.model.eval()
        self.mode = "eval"

def to(self, device):
    """Set a computational device (eg. cpu or cuda).

    Args:
        device (str): A computational device to be computed on.

    """
    self.device = device
    self.model.to(device)
    self.text_encoder.to(device)

def set_dataset(self, train_dataloader, val_dataloader, resolution=None):
    """Set a new dataset.

    Args:
        train_dataloader (torch.utils.data.DataLoader): A new training set.
        val_dataloader (torch.utils.data.DataLoader): A new validation set.
        resolution (Tuple[int], optional): A new resolution of the images in the given dataset. If None
        it was set to the size of the first image. Default to None.

    """
    self.train_dataloader = train_dataloader
    self.val_dataloader = val_dataloader
    self.resolution = next(iter(train_dataloader))[0].shape[-2:] if resolution == None else resolution

def parameters(self):
    return [p for p in self.model.parameters() if p.requires_grad]

```

```

In [ ]: text_encoder = CLIPTextModel.from_pretrained("CompVis/stable-diffusion-v1-4", subfolder="text_encoder", re

text_encoder.requires_grad_(False)

trainer_config = {
    "model": model,
    "tokenizer": tokenizer,
    "text_encoder": text_encoder,

```

```
"train_dataloader": train_dataloader,  
"resolution": RESOLUTION,  
"val_dataloader": test_dataloader,  
"device": "cuda",  
"ckpt_path": "./weights/",  
"lr": 3e-4,  
}  
  
trainer = Trainer(**trainer_config)
```

```
In [ ]: epoch = 10  
train_losses, val_losses = trainer.fit(epoch)
```

```
In [ ]: plt.plot(range(epoch), train_losses, label="training loss")  
plt.plot(range(epoch), val_losses, label="validation loss")  
plt.legend()  
plt.show()
```

Inference

Although the diffusion model is not trained to color the digit 5-9, it can generate the colored digit 5-9, achieving zero-shot generation capability.

```
In [ ]: fig, axs = plt.subplots(ncols=10, nrows=10, figsize=(10, 10))
fig.tight_layout(pad=2)
text_label = ["zero", "one", "two", "three", "four", "five", "six", "seven", "eight", "nine"]

generated_images = []
for iter in range(10):
    mod_text_label = list(map(lambda x: " ".join([x, random.choice(list(COLOR.keys()))]), text_label))
    input_ids = tokenizer(mod_text_label, padding="max_length", truncation=True, max_length=tokenizer.model_max_length)
    context = text_encoder(input_ids, return_dict=False)[0]
    sampled_images = trainer.model.sample(batch_size = 10, context=context)#
    generated_images.append(sampled_images)
    for idx_sampler in range(10):
        axs[iter][idx_sampler].imshow(rearrange(unnormalize_to_zero_to_one(sampled_images[idx_sampler]), "cwh"))
        axs[iter][idx_sampler].set_axis_off()
        axs[iter][idx_sampler].set_title(mod_text_label[idx_sampler])
plt.show()
```

OT 4. Does our model have any other zero-shot generation capability?

Answer here

Hint: Is it possible to generate other colors?

Conclusion

Congratulations! You have successfully built the DDPM and Low-Rank Adaptation. Fortunately, in the real world, there are pre-built libraries ready to be used without implementing both from scratch. For diffusion, we have `diffusers` library providing all the necessary modules for the diffusion pipeline, as well as LoRA, we can use `peft` library to inject LoRA into the model by declaring injected modules in the LoRAConfig. For more details, please visit the HuggingFace website.